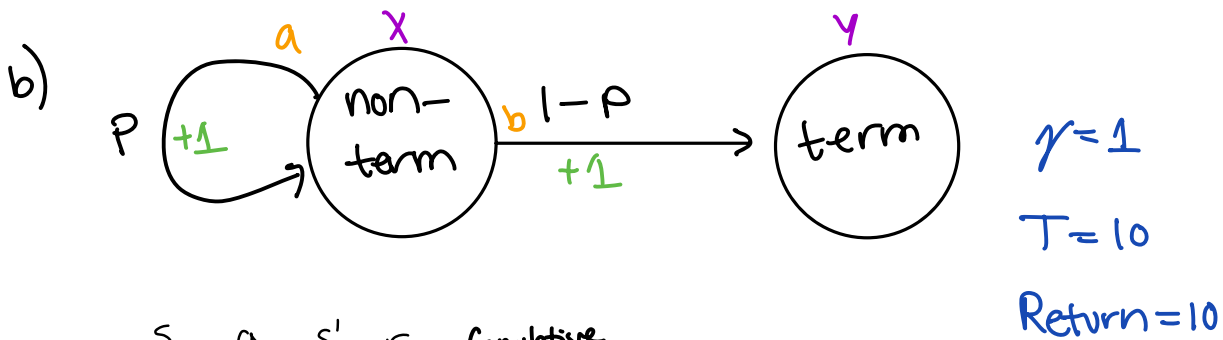


1. a) For a game like Black Jack, using first-visit or every-visit MC will both result in the same final policy since it is never possible in one episode to return to a state that you have already been to. That would require cards with non-positive values which don't exist in the game. In a sense, the first visit is always the ONLY visit.



	S	a	S'	r	Cumulative
1	x	a	x	1	10
2	x	a	x	1	9
3	x	a	x	1	8
$\vdots$		$\vdots$			$\vdots$
10	x	b	y	1	1

First Visit MC:

if we follow first-visit rules

$$V(x) = \frac{10}{1} = 10$$

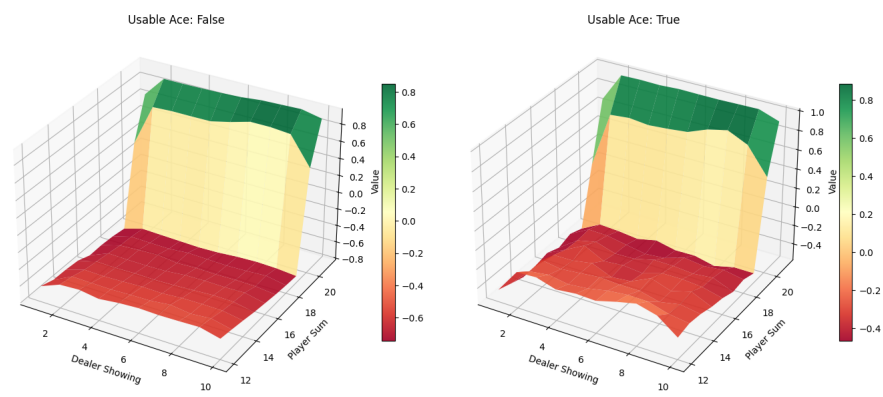
Every Visit MC

if we follow every-visit MC

$$V(x) = \frac{10 + 9 + 8 + 7 + 6 + 5 + 4 + 3 + 2 + 1}{10} = 5.5$$

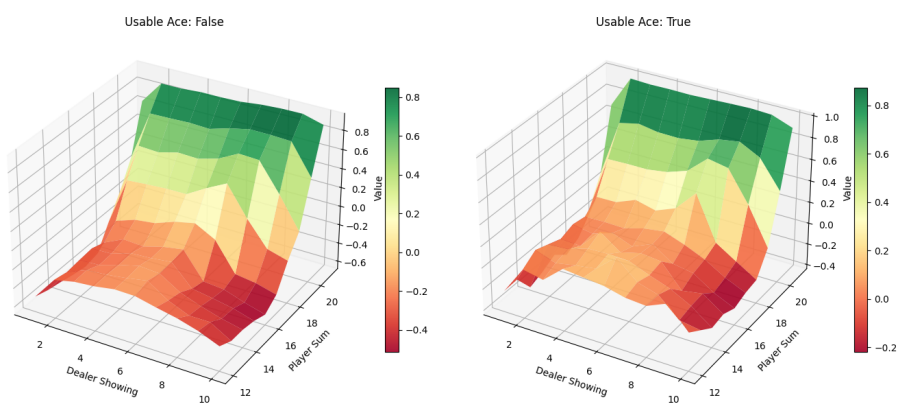
2 a) Fixed Policy MC Evaluation

Value Function for Blackjack (Fixed Policy)

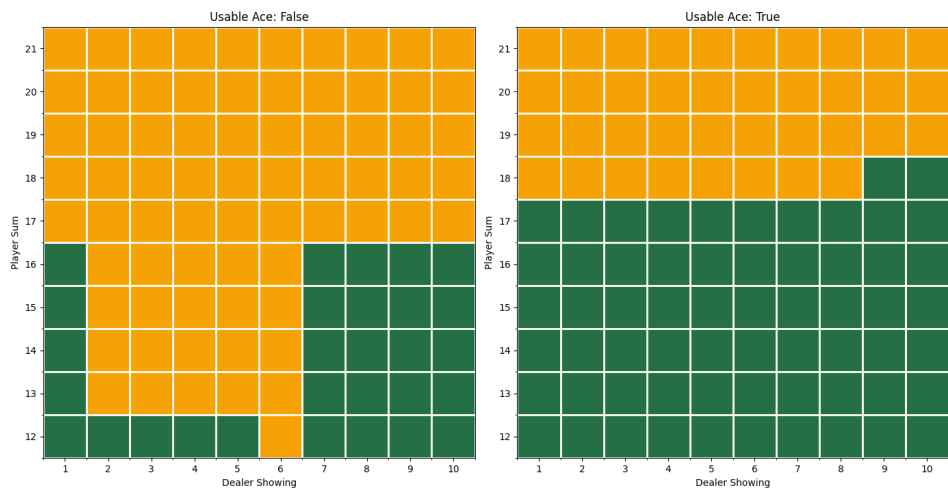


2 b) MC-Control w/ Exploring Starts

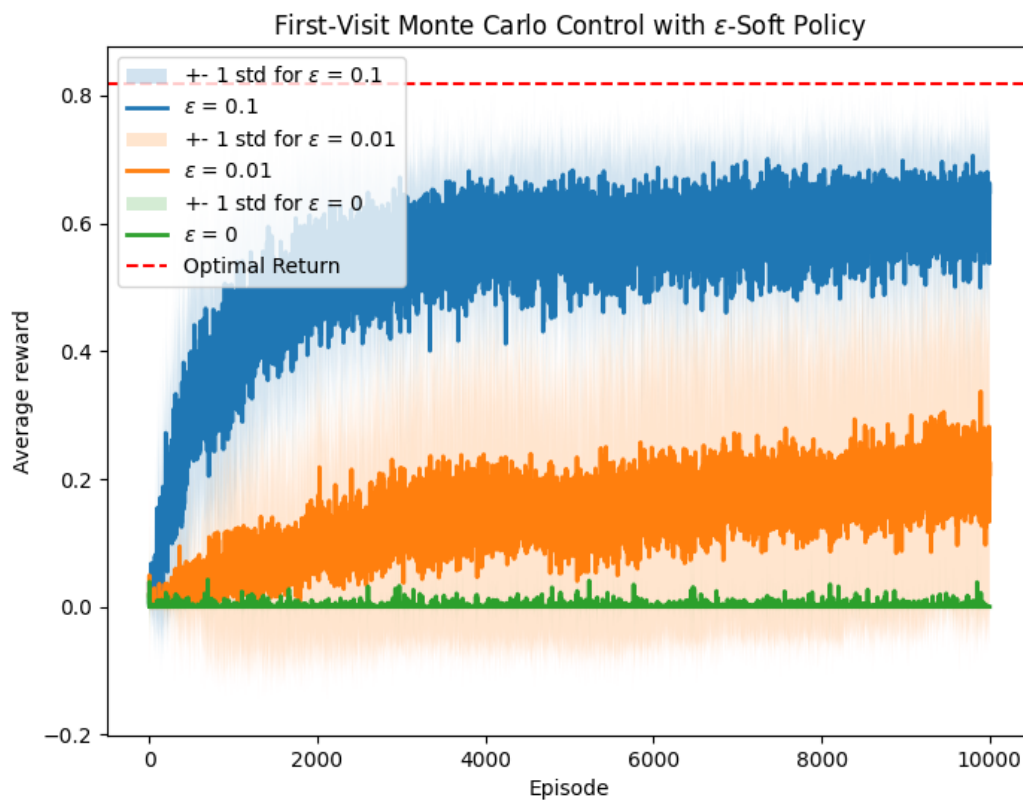
Value Function for Blackjack (Optimal Policy)



Optimal Policy for Blackjack (Green=Hit, Orange=Stick)



3) Four Rooms



b) When $\epsilon=0$, $\pi(s|a)$ becomes a greedy and deterministic policy. Additionally, the agent always starts in the same state. These two facts together mean that there is insufficient coverage, mainly there exists a set of actions, a and states, s , where $\pi(a|s)=0$. Without this coverage and exploration, the policy is not guaranteed to converge to an optimal policy.

When $\epsilon=0$, exploring starts are necessary because it introduces variability into the episodes which forces new actions to be taken which allows for coverage and exploration and allows for the guarantee of convergence.

4)

a)

$$V_{n+1} = \frac{\sum_{k=1}^n W_k G_k}{\sum_{k=1}^n W_k}$$

$$V_{n+1} = \frac{1}{\sum_{k=1}^n W_k} \left(W_n G_n + \sum_{k=1}^{n-1} W_k G_k \right)$$

$$\text{Let } C_n = \sum_{k=1}^n W_k$$

$$C_{n-1} = \sum_{k=1}^{n-1} W_k$$

$$V_{n+1} = \frac{1}{C_n} \left(W_n G_n + \frac{C_{n-1}}{C_{n-1}} \cdot \sum_{k=1}^{n-1} W_k G_k \right)$$

$$V_n = \frac{\sum_{k=1}^{n-1} W_k G_k}{\sum_{k=1}^{n-1} W_k}$$

$$V_{n+1} = \frac{1}{C_n} \left(W_n G_n + C_{n-1} V_n \right)$$

$$= \frac{\sum_{k=1}^{n-1} W_k G_k}{C_{n-1}}$$

$$C_{n-1} = C_n - W_n$$

$$V_{n+1} = \frac{1}{C_n} \left(W_n G_n + (C_n - W_n) V_n \right)$$

$$V_{n+1} = \frac{1}{C_n} \left(W_n G_n + C_n V_n - W_n V_n \right)$$

$$V_{n+1} = V_n + \frac{W_n G_n}{C_n} - \frac{W_n V_n}{C_n}$$

$$V_{n+1} = V_n + \frac{W_n}{C_n} (G_n - V_n)$$

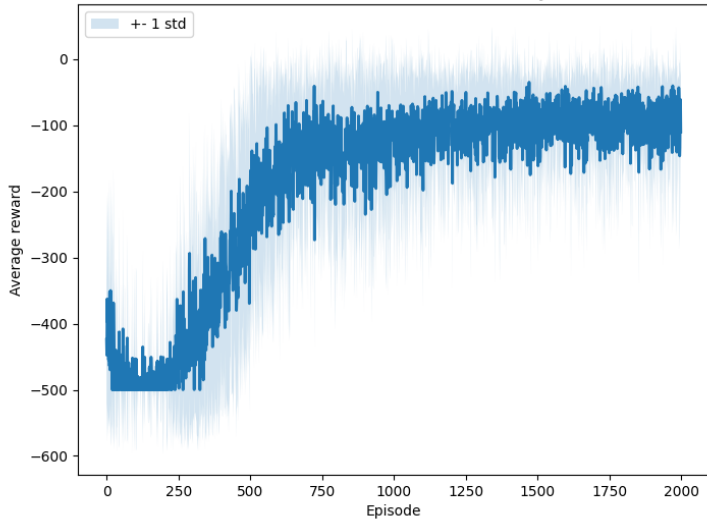
b) The W update step only involves $\frac{1}{b(A_t|S_t)}$
and not $\frac{\pi(A_t|S_t)}{b(A_t|S_t)}$ because π , the target policy,
is supposed to be a greedy policy and therefore is always
deterministic. ie. $\pi(A_t|S_t)=0$ or $\pi(A_t|S_t)=1$ and
nothing inbetween.

When $\pi(A_t|S_t)=0$, then the weight for that state
action pair is unchanged and the weight update step is
skipped entirely.

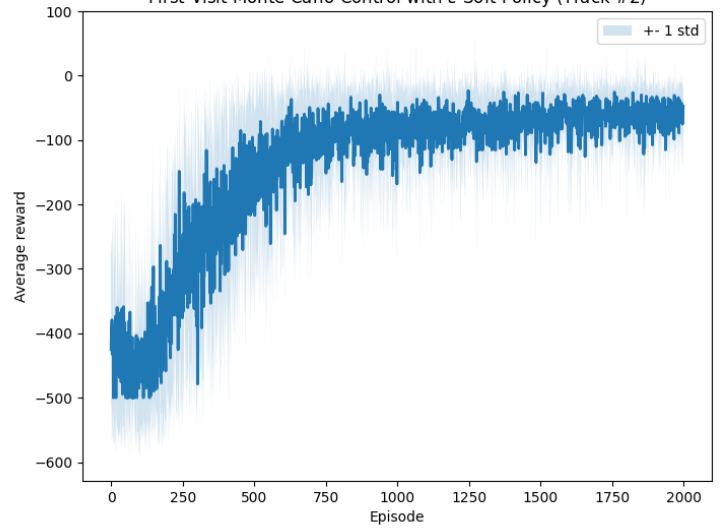
When $\pi(A_t|S_t)=1$, the weight for that action pair is
updated with $\frac{1}{b(A_t|S_t)}$

5) a) ϵ -soft

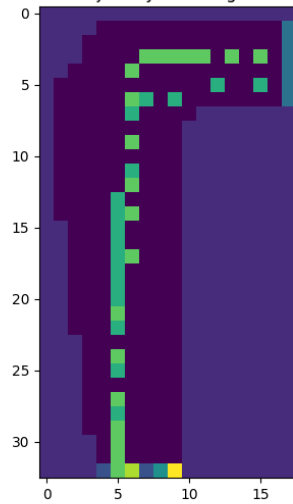
First-Visit Monte Carlo Control with ϵ -Soft Policy (Track #1)



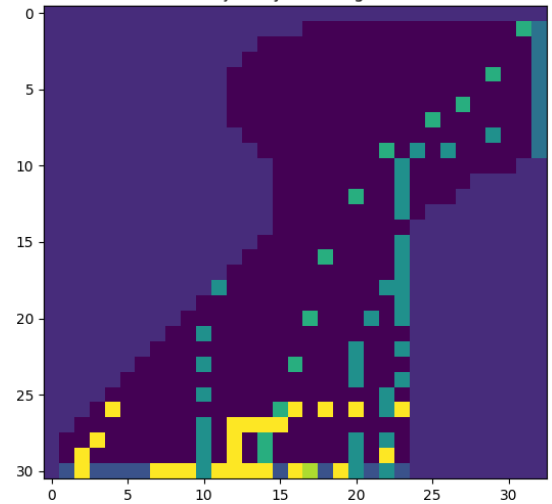
First-Visit Monte Carlo Control with ϵ -Soft Policy (Track #2)



Trajectory of the agent

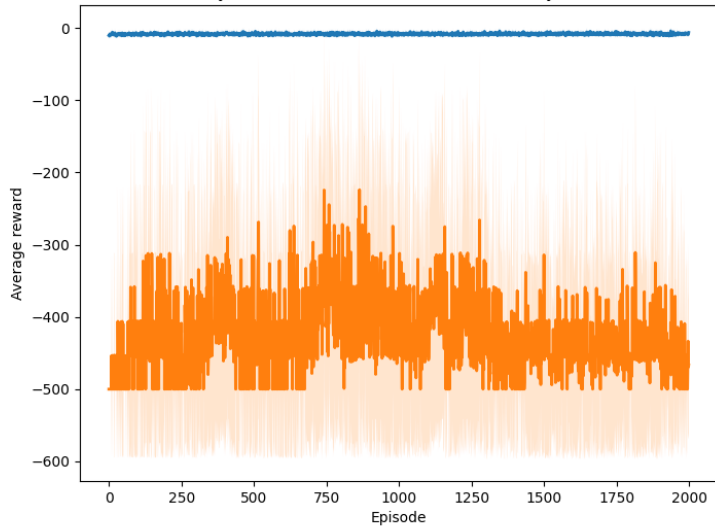


Trajectory of the agent

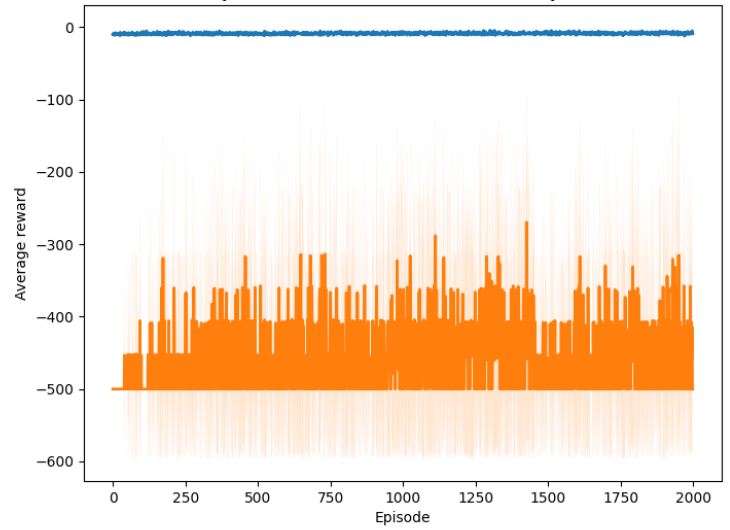


b) Off-Policy MC-Control

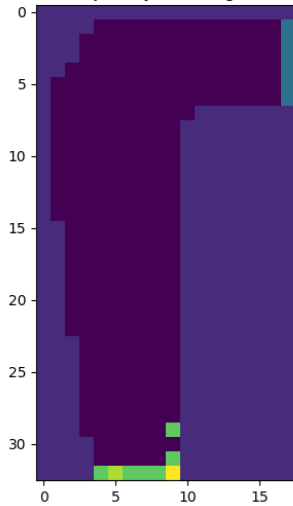
Off-Policy Monte Carlo Control with ϵ -Soft Policy (Track #1)



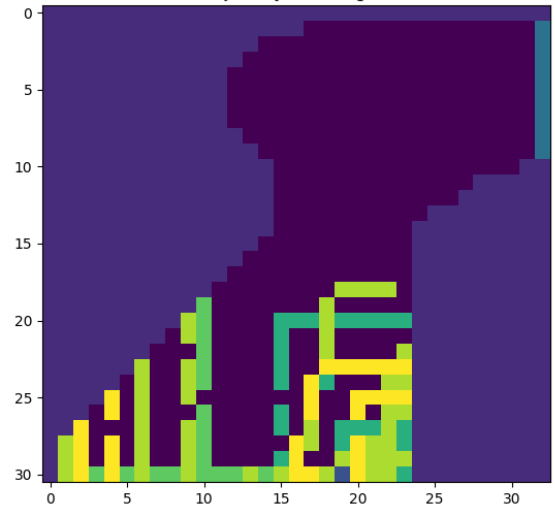
Off-Policy Monte Carlo Control with ϵ -Soft Policy (Track #2)



Trajectory of the agent



Trajectory of the agent



c) For my implementations it seems that the on-policy method resulted in much higher rewards and converged after far fewer episodes. However, in theory if given enough time to train, the off-policy method should converge to a better solution since the final target policy is more greedy. However training w/ off policy was much less efficient due to the large episode size and the high chance of $A_t \neq \pi(S_t)$ resulting in entire episodes being completely ignored.

One interesting difference between the racetracks is that it seems that for track #2, there seem to be a lot more resets due to the "car" crashing into the left wall when it drives completely vertical into it.

The agent seems to gravitate towards vertical and horizontal movements which works ok for Track #1 but results in a bit more of a struggle for track #2.